Министерство науки и высшего образования Российской Федерации

Муромский институт (филиал)

Федерального государственного бюджетного образовательного учреждения высшего образования

«Владимирский государственный университет имени Александра Григорьевича и Николая Григорьевича Столетовых»

Факультет ФИТР

Кафедра ПИН

КУРСОВАЯ РАБОТА

по дисциплине «Системы управления базами данных»

Тема «АРМ администратора гостиницы»

Оценка _____

Члены комиссии

_____

_____

Руководитель

Доц. каф. ПИН

Колпаков А. А.

Студент ПИНз-123

Терехов С. М.

Подпись _____

Муром 2026

Аннотация

В курсовой работе разработана автоматизированная информационная система «АРМ администратора гостиницы» для гостиницы Volga Grand Hotel. Приложение автоматизирует учёт номерного фонда и гостей, поиск свободных номеров, оформление бронирования и заселения, регистрацию дополнительных услуг, расчёт оплаты и перенос сведений о завершённом проживании в архив. Программная реализация выполнена на C# для платформы .NET 8 с использованием Windows Forms. Данные хранятся в локальной реляционной базе SQLite, которая создаётся и заполняется при первом запуске программы.

Проектирование системы выполнено последовательно. Диаграмма вариантов использования определяет участников и доступные им функции, DFD показывает информационные потоки между клиентом, администратором, платёжной системой и хранилищами, UML-диаграмма классов фиксирует объектную модель предметной области. Эти результаты трёх лабораторных работ согласованы с физической схемой базы данных и фактически реализованными модулями приложения. В базе созданы 12 таблиц и 12 триггеров; демонстрационный набор включает 20 номеров с изображениями BLOB и 88 записей в основных операционных таблицах.

Ключевые слова: гостиница, автоматизированное рабочее место, база данных, SQLite, C#, Windows Forms, UML, DFD, бронирование, заселение, оплата.

Abstract

The course project presents an automated workplace for the administrator of Volga Grand Hotel. The desktop application manages rooms and guests, searches room availability, creates bookings, registers check-in and check-out, records extra services, accepts payments and stores completed stays in an archive. The software is implemented in C# for .NET 8 with Windows Forms. A local SQLite database is created and populated automatically on the first launch.

The design combines the results of three laboratory assignments. The use-case diagram defines system actors and their operations, the data-flow diagrams describe information exchange and persistent stores, and the UML class diagram represents the domain model. These models are mapped to the implemented database schema and application modules. The database contains 12 tables and 12 triggers; the demonstration data set includes 20 rooms with BLOB images and 88 records in the principal operational tables.

Keywords: hotel, administrator workplace, database, SQLite, C#, Windows Forms, UML, DFD, booking, check-in, payment.

Содержание

Аннотация 2

Abstract 3

Введение 6

1 Анализ технического задания 7

1.1 Предметная область и границы системы 7

1.2 Функциональные и нефункциональные требования 8

1.3 Варианты использования и спецификации процессов 8

2 Моделирование потоков данных 10

2.1 Контекстная DFD 10

2.2 Декомпозиция процессов уровня 0 10

2.3 Согласование DFD с реализованной системой 12

3 Проектирование объектной модели и базы данных 13

3.1 UML диаграмма классов 13

3.2 Концептуальная модель данных 13

3.3 Логическая модель данных 14

3.4 Физическая модель SQLite 15

4 Выбор технологий и архитектуры 16

4.1 Платформа C Sharp и Windows Forms 16

4.2 СУБД SQLite и доступ к данным 16

4.3 Структура проекта 16

5 Реализация основных алгоритмов 17

5.1 Инициализация базы и демонстрационных данных 17

5.2 Поиск свободных номеров 17

5.3 Бронирование и заселение 17

5.4 Расчёт оплаты и архивирование . . . 18

5.5 Ограничения целостности и журнал аудита . . . 18

6 Описание пользовательского интерфейса . . . 19

7 Тестирование системы . . . 22

Заключение . . . 23

Список использованной литературы . . . 24

Приложение А Диаграммы проектирования . . . 25

Приложение Б Фрагменты текста программы . . . 31

Б.1 Файл Program.cs . . . 31

Б.2 Файл HotelAdminArm.csproj . . . 32

Б.3 Физическая схема и триггеры Database.Schema.cs . . . 33

Б.4 Фрагменты поиска, бронирования и заселения . . . 40

Б.5 Фрагменты расчёта и оплаты . . . 43

Введение

Работа администратора гостиницы связана с постоянным изменением сведений о номерах, гостях и периодах проживания. При ручном учёте трудно одновременно контролировать доступность комнаты, корректность дат, состав оказанных услуг и итоговую сумму к оплате. Ошибка в одном из этих элементов может привести к пересекающимся бронированиям, неверному статусу номера или потере истории обслуживания клиента. Поэтому для стойки регистрации требуется единая информационная система, в которой операции над данными выполняются по согласованным правилам.

Объектом исследования является процесс административного обслуживания гостей гостиницы. Предмет исследования составляет программная и информационная модель автоматизированного рабочего места, поддерживающего бронирование, заселение, проживание, оплату и архивирование данных.

Цель курсовой работы состоит в разработке настольной автоматизированной информационной системы для администратора гостиницы с локальной реляционной базой данных. Система должна реализовать требования задания и сохранять целостность информации при выполнении основных операций.

Для достижения цели решены следующие задачи:

1. проанализирована предметная область и определены границы автоматизации;
2. сформулированы функциональные и нефункциональные требования;
3. построены диаграмма вариантов использования, контекстная DFD, DFD уровня 0 и UML-диаграмма классов;
4. разработаны концептуальная, логическая и физическая модели данных;
5. реализована база SQLite с ограничениями, внешними ключами и триггерами;
6. создано приложение Windows Forms для выполнения операций администратора;
7. подготовлен демонстрационный набор данных и проведено функциональное тестирование.

В работе применены методы анализа требований, функционального и объектного моделирования, проектирования реляционных баз данных, нормализации отношений и функционального тестирования. Пояснительная записка построена так, чтобы проследить переход от моделей лабораторных работ к структуре базы данных и программной реализации.

1 Анализ технического задания

1.1 Предметная область и границы системы

Гостиница располагает номерами различной категории, вместимости и стоимости. В исходном задании выделены люкс, полулюкс, одноместные и двухместные номера. Тариф устанавливается за номер в сутки и не зависит от количества проживающих. Сведения о категории, цене, вместимости, этаже, статусе и изображении номера должны храниться в базе данных и быть доступны администратору для просмотра и изменения.

Основной пользователь системы - администратор гостиницы. Он принимает заявку по телефону, вводит критерии поиска, выбирает свободный номер и оформляет бронь. При прибытии гостя администратор проверяет сведения, заполняет регистрационную карточку и создаёт запись о проживании. В период проживания к карточке могут добавляться платные услуги. При выезде приложение рассчитывает стоимость комнаты и услуг, фиксирует платёж, освобождает номер и переносит сведения о завершённом проживании в архив.

Клиент рассматривается как внешний участник процессов: он сообщает требования к номеру и персональные данные, получает подтверждение бронирования, проживает в выбранном номере и оплачивает услуги. Самостоятельный клиентский интерфейс в текущей версии не реализован. Внешняя платёжная система присутствует в DFD как логический участник безналичной операции; фактическая учебная реализация регистрирует результат оплаты локально без интеграции с банковским шлюзом.

В границы системы включены операции с номерным фондом, гостями, бронями, проживанием, услугами, платежами, архивом и журналом изменений. Управление персоналом, бухгалтерский учёт организации, уборка помещений, закупки и техническое обслуживание оборудования не автоматизируются.

1.2 Функциональные и нефункциональные требования

Код	Требование	Критерий выполнения	Реализация
F1	Ведение справочника номеров	Добавление, изменение и удаление номера; хранение типа, цены, вместимости, статуса и фотографии	Вкладка Номера; таблицы RoomTypes и Rooms
F2	Учёт гостей	Создание и изменение карточки с ФИО, паспортом, контактами и адресом	Вкладка Гости; таблица Guests
F3	Поиск доступности	Отбор свободных номеров по периоду, типу и минимальной вместимости	Вкладка Бронирование; запрос к Rooms, Bookings и Stays
F4	Бронирование	Создание, подтверждение, изменение, отмена и удаление брони	Bookings и триггеры проверки дат
F5	Заселение	Создание проживания по брони или без предварительной брони	Stays; изменение статуса Rooms
F6	Дополнительные услуги	Изменение справочника услуг и привязка услуги к проживанию	Services и ServiceOrders
F7	Оплата	Расчёт комнаты и услуг, один платёж за несколько проживаний	Payments и PaymentStays
F8	Выезд и архив	Закрытие проживания, освобождение номера и сохранение архивной записи	GuestArchive и trg_stays_after_update
F9	Аудит	Фиксация ключевых операций изменения данных	AuditLog и триггеры аудита
F10	Демонстрационные данные	Не менее 50 записей и изображения по тематике проекта	Seed; 88 основных записей и 20 BLOB-фотографий

К нефункциональным требованиям относятся автономность работы, простой запуск без отдельного сервера, понятный интерфейс, защита от некорректных периодов и пересекающихся размещений, параметризованное выполнение SQL-команд, обработка исключений и сохранение данных между запусками. Эти требования определили выбор локальной SQLite и настольного интерфейса Windows Forms.

1.3 Варианты использования и спецификации процессов

Диаграмма вариантов использования из лабораторной работы №1 показывает функции в терминах целей участников. Клиент инициирует поиск и бронирование. Администратор работает с бронями, регистрирует заселение, оформляет выезд и расчёт, а также ведёт номерной фонд. Служебные операции включают проверку полномочий как обязательный шаг концептуальной модели. В текущем учебном приложении отдельный модуль авторизации не реализован, поэтому диаграмма задаёт направление дальнейшего развития, а остальные прецеденты соответствуют действующим вкладкам.

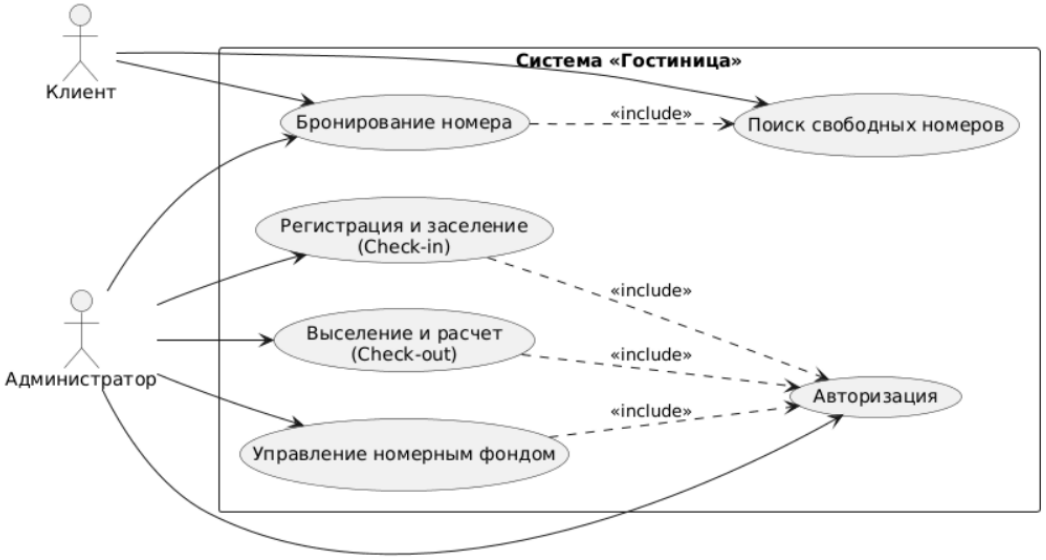


Рисунок 1 - Диаграмма вариантов использования системы «Гостиница»

Прецедент «Бронирование номера» начинается с ввода дат, категории и требуемой вместимости. Система исключает номера, для которых найдено пересечение с активной бронью или проживанием, после чего администратор выбирает гостя и номер и сохраняет запись. Если дата выезда не позже даты заезда либо период занят, операция прекращается с сообщением об ошибке.

Прецедент «Регистрация и заселение» выполняется по подтверждённой брони или без неё. Администратор выбирает гостя и свободный номер, задаёт фактическую дату заезда и плановую дату выезда. После сохранения создаётся запись Stays, а триггер переводит номер в состояние «Занят». При заселении по брони статус связанной записи Bookings изменяется на «Заселен».

Прецедент «Выселение и расчёт» включает определение количества суток, суммирование стоимости дополнительных услуг, регистрацию одного платежа и связей с оплачиваемыми проживаниями. После подтверждения транзакции записи проживания переводятся в состояние «Выбыл», номер освобождается, а сведения о госте и периоде помещаются в архив.

2 Моделирование потоков данных

2.1 Контекстная DFD

Контекстная диаграмма из лабораторной работы №2 представляет систему одним процессом «Управлять гостиницей». Клиент передаёт критерии поиска и сведения для бронирования, а получает доступные варианты и подтверждение. Администратор вводит данные заезда, выезда и номерного фонда и получает статусы, счета и отчётные сведения. Платёжная система принимает запрос на безналичную операцию и возвращает результат.

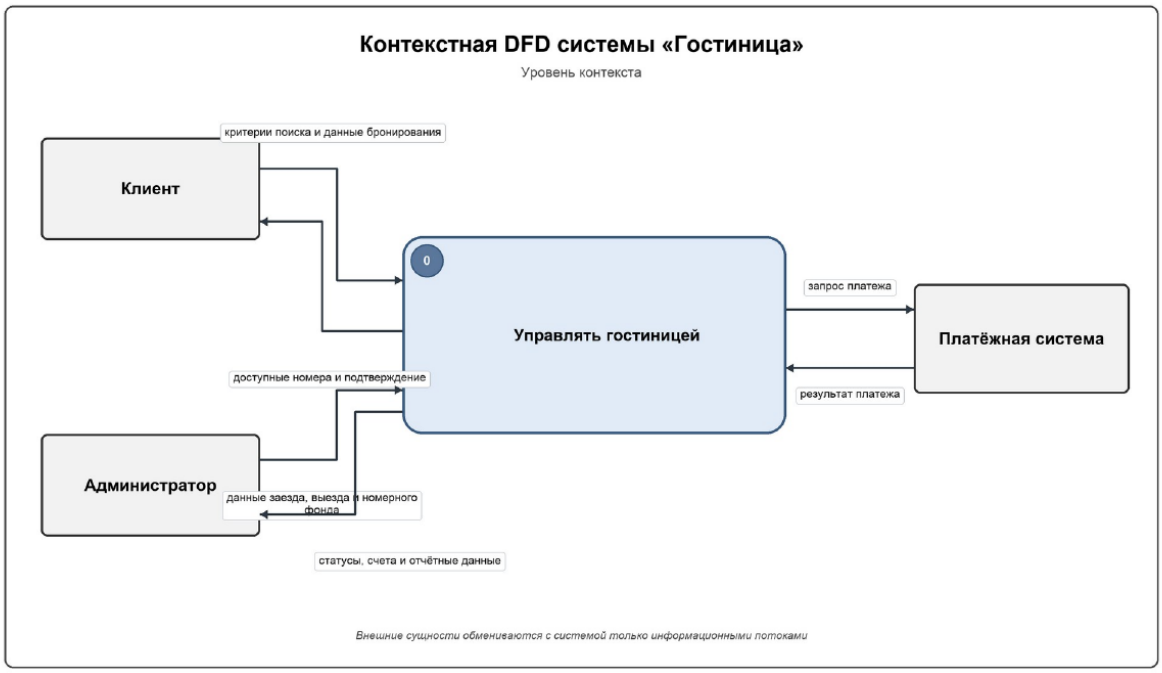


Рисунок 2 - Контекстная DFD системы «Гостиница»

Контекстный уровень фиксирует границу автоматизации и не раскрывает внутреннее хранение. Это позволяет проверить, что все внешние участники и обмены учтены до перехода к декомпозиции.

2.2 Декомпозиция процессов уровня 0

На уровне 0 основной процесс разделён на четыре функции: оформление бронирования, регистрация заселения, оформление выезда и оплаты, ведение номерного фонда. Между ними используются пять базовых хранилищ DFD: клиенты, номерной фонд, бронирования, проживания и платежи.

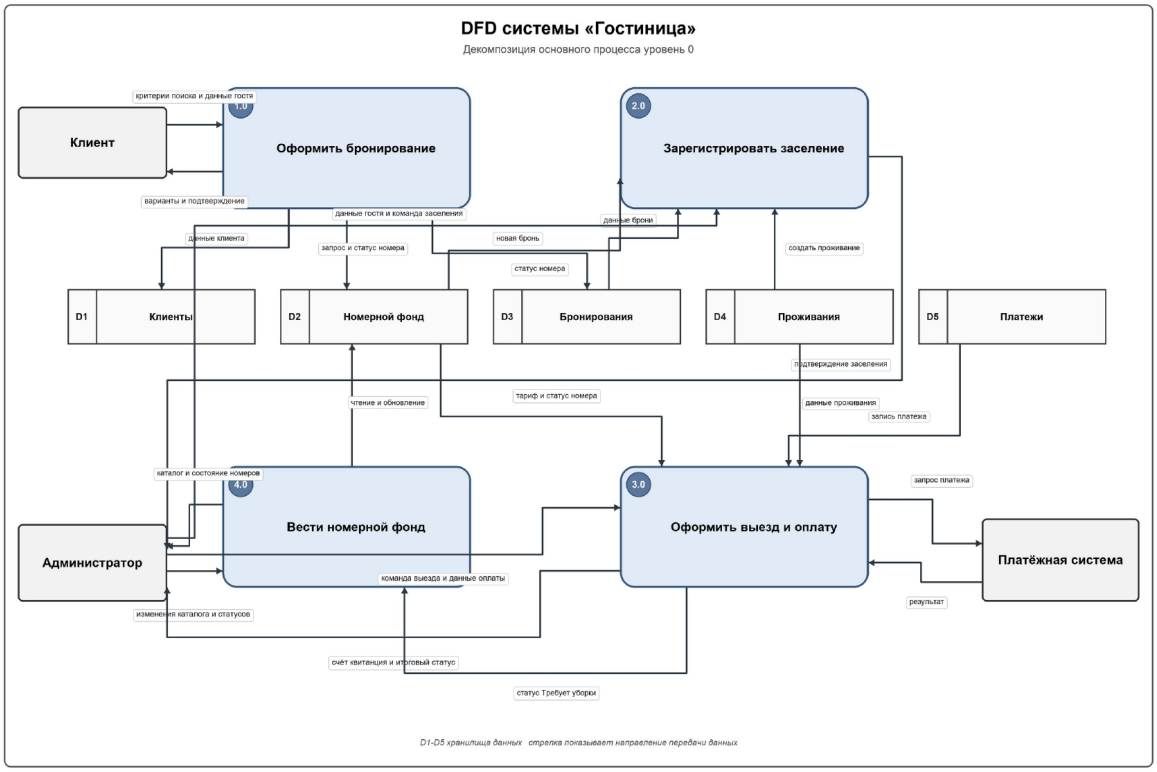


Рисунок 2 - DFD уровня 0 системы «Гостиница»

Рисунок 3 - DFD уровня 0 системы «Гостиница»

Номер	Процесс	Входные данные	Результат
1.0	Оформить бронирование	Критерии поиска, данные гостя	Запись брони, подтверждение и варианты номеров
2.0	Зарегистрировать заселение	Бронь или данные гостя, выбранный номер	Запись проживания и занятый статус номера
3.0	Оформить выезд и оплату	Проживания, услуги, способ оплаты	Платёж, архивная запись и освобождение номера
4.0	Вести номерной фонд	Характеристики, тариф, статус, изображение	Актуальный каталог номеров

2.3 Согласование DFD с реализованной системой

Элемент DFD	Таблицы SQLite	Назначение
D1 Клиенты	Guests	Хранит карточки гостей и используется во всех клиентских операциях
D2 Номерной фонд	RoomTypes, Rooms	Типы, характеристики, цены, статусы и BLOB-изображения
D3 Бронирования	Bookings	Планируемые периоды, гость, номер и состояние брони
D4 Проживания	Stays	Фактический заезд, плановый и фактический выезд, состояние
D5 Платежи	Payments, PaymentStays	Плательщик, сумма, способ и состав оплаченных проживаний
Дополнительные данные	Services, ServiceOrders	Справочник услуг и факты их оказания
Архив и аудит	GuestArchive, AuditLog	История выездов и журнал изменений

Балансировка между контекстной диаграммой и уровнем 0 соблюдена: входы и выходы внешних сущностей раскрываются внутри четырёх процессов, новые внешние участники не появляются. Реализация расширяет исходные пять хранилищ, добавляя услуги, архив и аудит. Такое расширение не меняет внешние потоки и уточняет внутреннюю структуру данных.

3 Проектирование объектной модели и базы данных

3.1 UML диаграмма классов

UML-диаграмма из лабораторной работы №3 описывает статическую модель предметной области. Абстрактный пользователь объединяет общие сведения о клиенте и администраторе. Бронирование связывает клиента с номером на заданный период; состоявшаяся бронь может породить одно проживание. Проживание относится к номеру и участвует в расчёте платежей. Кратности фиксируют обязательность клиента и номера для каждой брони и возможность повторного использования номера в разные периоды.

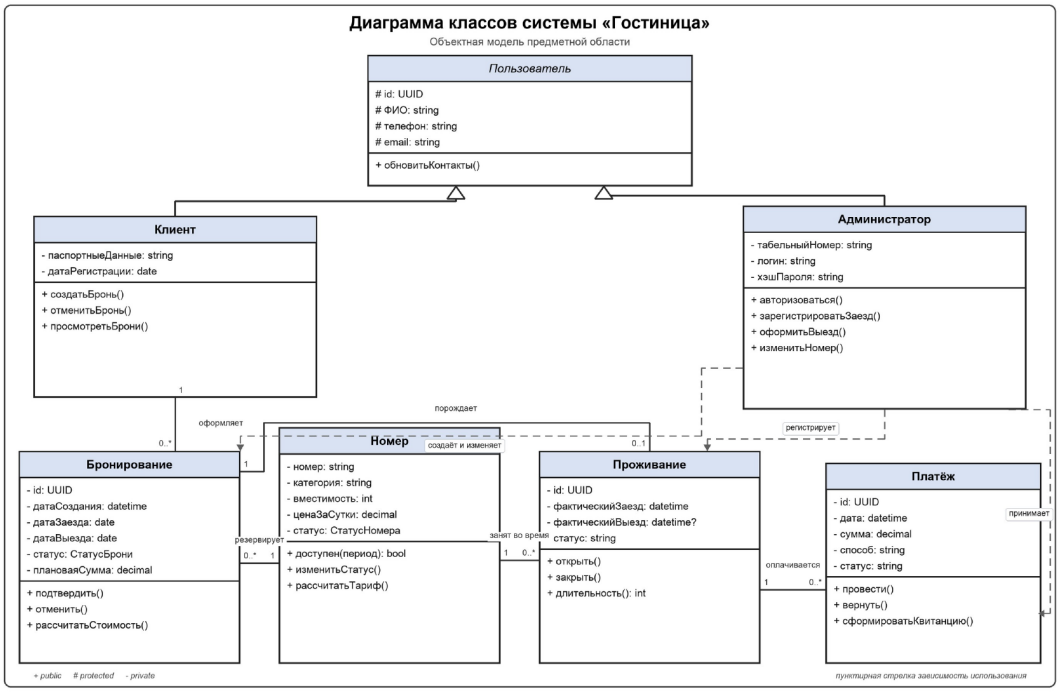


Рисунок 1 - Диаграмма классов системы «Гостиница»

Рисунок 4 - UML-диаграмма классов системы «Гостиница»

Диаграмма классов является концептуальной объектной моделью и не повторяет структуру исходного кода буквально. В приложении операции сосредоточены в частичных классах MainForm и статическом модуле Database, а предметные объекты представлены строками таблиц и элементами интерфейса. Соответствие обеспечивается на уровне данных и сценариев: классам Клиент, Номер, Бронирование, Проживание и Платёж отвечают таблицы Guests, Rooms, Bookings, Stays и Payments.

3.2 Концептуальная модель данных

Концептуальная модель выделяет устойчивые сущности, необходимые для работы гостиницы, и не зависит от выбранной СУБД. Гость оформляет бронь на номер. Подтверждённая бронь используется при создании проживания, однако допускается заселение без предварительной брони. К проживанию добавляются услуги. Оплата закрывает одно или несколько проживаний и выполняется выбранным плательщиком.



Рисунок 5 - Концептуальная модель данных

3.3 Логическая модель данных

При переходе к логической модели категории номеров вынесены в RoomTypes, а факты оказания услуг - в ServiceOrders. Связь между платежами и проживанием имеет вид многие ко многим, поскольку один платёж может закрывать несколько номеров, а структура допускает фиксацию нескольких финансовых операций для одного проживания. Для её реализации введена таблица PaymentStays с составным первичным ключом.

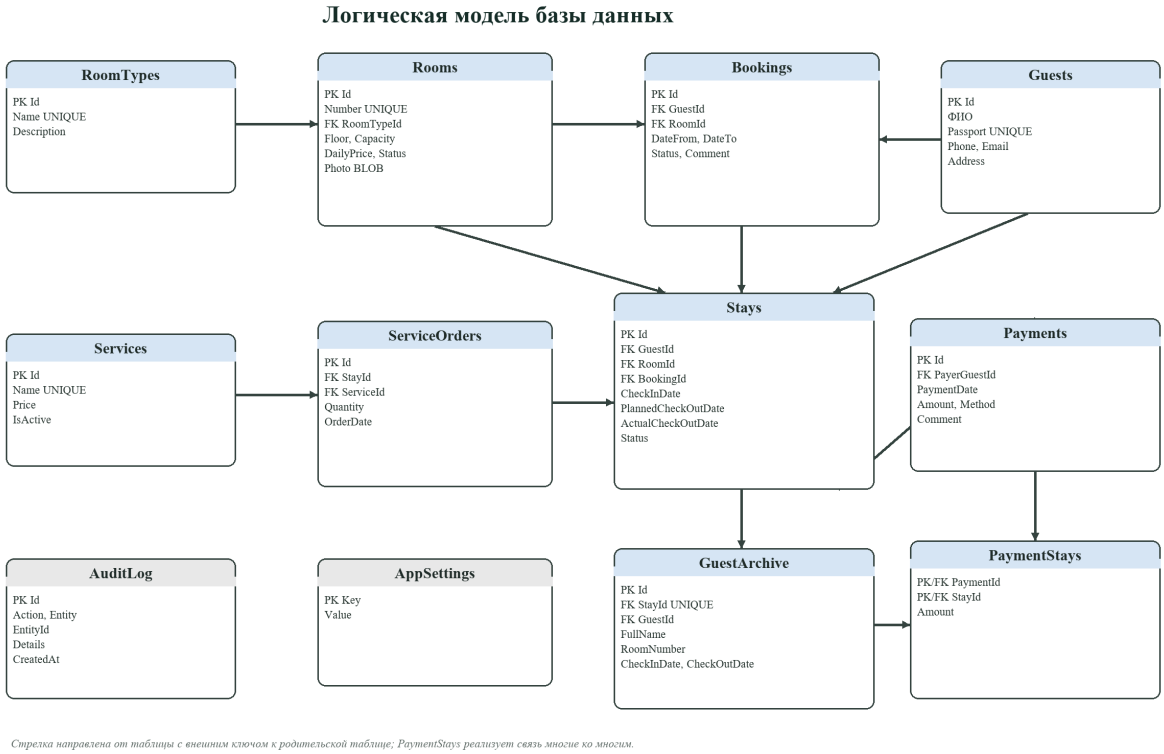


Рисунок 6 - Логическая модель базы данных

Отношения приведены к форме, исключающей повторение справочных значений. Тип номера и услуга хранятся один раз, а операционные таблицы содержат внешние ключи. Архив сохраняет денормализованные ФИО и номер комнаты намеренно: эти значения фиксируют состояние на момент выезда и позволяют получить историю независимо от последующих изменений карточки гостя или номера.

3.4 Физическая модель SQLite

Таблица	Первичный ключ	Основные поля	Назначение
RoomTypes	Id INTEGER	Name UNIQUE, Description	Справочник типов номеров
Rooms	Id INTEGER	RoomTypeId, Number UNIQUE, Floor, Capacity, DailyPrice, Status, Photo BLOB	Номерной фонд
Guests	Id INTEGER	Passport UNIQUE, ФИО, контакты, адрес	Карточки гостей
Bookings	Id INTEGER	GuestId, RoomId, DateFrom, DateTo, Status	Бронирования
Stays	Id INTEGER	GuestId, RoomId, BookingId, даты, Status	Проживания
Services	Id INTEGER	Name UNIQUE, Price, IsActive	Справочник услуг
ServiceOrders	Id INTEGER	StayId, ServiceId, Quantity, OrderDate	Услуги проживания
Payments	Id INTEGER	PayerGuestId, PaymentDate, Amount, Method	Платежи
PaymentStays	PaymentId + StayId	Amount	Состав платежа
GuestArchive	Id INTEGER	StayId UNIQUE, GuestId, FullName, RoomNumber, даты	Архив выездов
AuditLog	Id INTEGER	Action, Entity, EntityId, Details, CreatedAt	Журнал действий
AppSettings	Key TEXT	Value	Настройки и версия дизайна

Все внешние ключи включаются параметром ForeignKeys в строке подключения. Денежные значения хранятся как REAL с проверкой неотрицательности, даты - в текстовом формате ISO yyyy-MM-dd. Для статусов заданы ограничения CHECK. Фотография номера хранится в Rooms.Photo как BLOB. Поля CreatedAt заполняются выражением datetime('now', 'localtime').

4 Выбор технологий и архитектуры

4.1 Платформа C Sharp и Windows Forms

Приложение реализовано на языке C# и нацелено на net8.0-windows. Windows Forms подходит для учебного автоматизированного рабочего места, поскольку предоставляет готовые элементы таблиц, редакторов, вкладок и диалогов, работает непосредственно в Windows и не требует развёртывания веб-сервера. Частичные классы MainForm разделяют код по функциональным областям, сохраняя единую форму приложения.

Интерфейс формируется программно. Общие методы создают таблицы, поля ввода, кнопки и компоновочные панели. Класс HotelBranding содержит палитру, название гостиницы, логотип и генератор изображений комнат. Благодаря этому оформление не зависит от внешних файлов и применяется одинаково ко всем вкладкам.

4.2 СУБД SQLite и доступ к данным

SQLite выбрана как встраиваемая реляционная СУБД. База хранится в одном файле hotel.db в каталоге LocalApplicationData пользователя. Для запуска не требуется отдельная служба базы данных, создание учётной записи или настройка сетевого подключения. Это соответствует требованию автономности и упрощает перенос проекта между учебными компьютерами.

Доступ выполняется через пакет Microsoft.Data.Sqlite. Модуль Database формирует строку подключения, включает внешние ключи, выполняет команды и запросы, создаёт параметры и поддерживает транзакции. Пользовательские значения не объединяются со строкой SQL, а передаются параметрами, что исключает изменение структуры команды введённым текстом.

4.3 Структура проекта

Файл или каталог	Назначение
Program.cs	Точка входа, инициализация базы, запуск MainForm
HotelBranding.cs	Название, цвета, логотип, оформление и генерация фотографий комнат
Data/Database.cs	Подключение, запросы, параметры, транзакции и миграция дизайна
Data/Database.Schema.cs	DDL-описание 12 таблиц и 12 триггеров
Data/Database.Seed.cs	Создание демонстрационных данных
MainForm.cs	Главное окно, общие компоненты и вкладка сводки
MainForm.RoomsGuests.cs	Номера, фотографии и карточки гостей
MainForm.BookingsStays.cs	Поиск, бронирование, заселение и архив
MainForm.ServicesPayments.cs	Услуги, платежи и завершение проживания
Docs	Пояснительная записка, диаграммы, лабораторные работы и скриншоты

Архитектура является двухуровневой: слой представления объединён с прикладной логикой в WinForms-форме, а модуль Database инкапсулирует работу с хранилищем. Для учебного объёма такая организация остаётся понятной и обеспечивает разделение SQL-схемы, начальных данных и интерфейсных сценариев.

5 Реализация основных алгоритмов

5.1 Инициализация базы и демонстрационных данных

При запуске Program.Main вызывает Database.Initialize. Метод создаёт каталог для данных, открывает соединение и выполняет единый пакет DDL-команд. Если таблица Rooms пуста, в транзакции добавляются типы комнат, 20 номеров, 24 гостя, 10 услуг, 10 броней, 13 проживаний, 8 заказов услуг и 3 платежа. Для пяти завершённых проживаний формируется архив.

Каждому номеру соответствует изображение PNG, сформированное классом HotelBranding и записанное в поле Photo. После создания или открытия существующей базы выполняется проверка версии оформления. При необходимости обновляются название гостиницы и изображения, а номер версии сохраняется в AppSettings.

5.2 Поиск свободных номеров

Алгоритм поиска получает дату заезда, дату выезда, тип номера и минимальную вместимость. Сначала проверяется порядок дат. Затем выбираются номера требуемого типа и вместимости, для которых не существует активной брони с пересекающимся периодом и не существует проживания со статусом «Проживает» в пересекающемся интервале.

Пересечение интервалов проверяется условием DateFrom < существующая DateTo и DateTo > существующая DateFrom. Граничный случай, при котором новый заезд совпадает с датой предыдущего выезда, не считается пересечением. Та же логика повторена в триггерах Bookings и Stays, поэтому защита сохраняется даже при выполнении команды вне пользовательского интерфейса.

5.3 Бронирование и заселение

После выбора гостя и номера бронирование сохраняется в Bookings. Статус принимает значения «Новое», «Подтверждено», «Заселен» или «Отменено». Триггеры до вставки и обновления запрещают обратный период и пересечение с другой активной бронью или проживанием.

Заселение создаёт запись Stays. Поле BookingId заполняется при наличии брони, но допускает NULL для гостя, пришедшего без предварительной заявки. Триггер после вставки переводит номер в состояние «Занят» и пишет событие в AuditLog. При заселении по брони её статус обновляется, что сохраняет согласованность жизненного цикла.

5.4 Расчёт оплаты и архивирование

Администратор может выделить несколько активных проживаний. Для каждого из них вычисляется число суток как целая разность дат, но не менее единицы. Стоимость проживания равна произведению количества суток на DailyPrice. К ней прибавляется сумма ServiceOrders, рассчитанная как Quantity × Services.Price.

Операция оплаты выполняется в транзакции. Сначала создаётся запись Payments с выбранным плательщиком, общей суммой и способом. Затем для каждого проживания добавляется строка PaymentStays с его долей суммы, после чего статус Stays изменяется на «Выбыл» и записывается фактическая дата выезда. Если один шаг завершается ошибкой, изменения всей операции отменяются.

Триггер после обновления проживания освобождает номер, если в нём не осталось активных размещений, и создаёт строку GuestArchive. Архив содержит ФИО, номер комнаты и даты заезда и выезда. Такая последовательность обеспечивает одновременное финансовое закрытие и сохранение истории обслуживания.

5.5 Ограничения целостности и журнал аудита

Триггер	Контролируемое правило
trg_bookings_validate_insert / update	Проверка порядка дат и отсутствия пересекающихся броней и проживаний
trg_stays_validate_insert / update	Проверка периода и отсутствия другого активного проживания в номере
trg_rooms_after_insert / update	Запись создания и изменения номера в AuditLog
trg_bookings_after_insert / update	Аудит операций с бронированиями
trg_stays_after_insert	Статус номера «Занят» и запись факта заселения
trg_stays_after_update	Освобождение номера, перенос в архив и аудит
trg_services_after_update	Аудит изменения услуги
trg_payments_after_insert	Аудит принятого платежа

Ограничения PRIMARY KEY, UNIQUE, FOREIGN KEY и CHECK отвечают за локальную корректность строк и ссылок. Триггеры контролируют правила, затрагивающие несколько строк или таблиц. Транзакции используются для составных операций оплаты и начального заполнения. Совместное применение этих механизмов уменьшает зависимость целостности данных от порядка действий пользователя.

6 Описание пользовательского интерфейса

Главное окно оформлено как единое рабочее место. В верхней части выводятся название и логотип Volga Grand Hotel, назначение системы и текущая дата. Навигация организована вкладками «Сводка», «Номера», «Гости», «Бронирование», «Заселение и архив», «Услуги» и «Оплата». Табличные списки находятся рядом с редакторами, поэтому администратор видит текущие данные и форму операции одновременно.

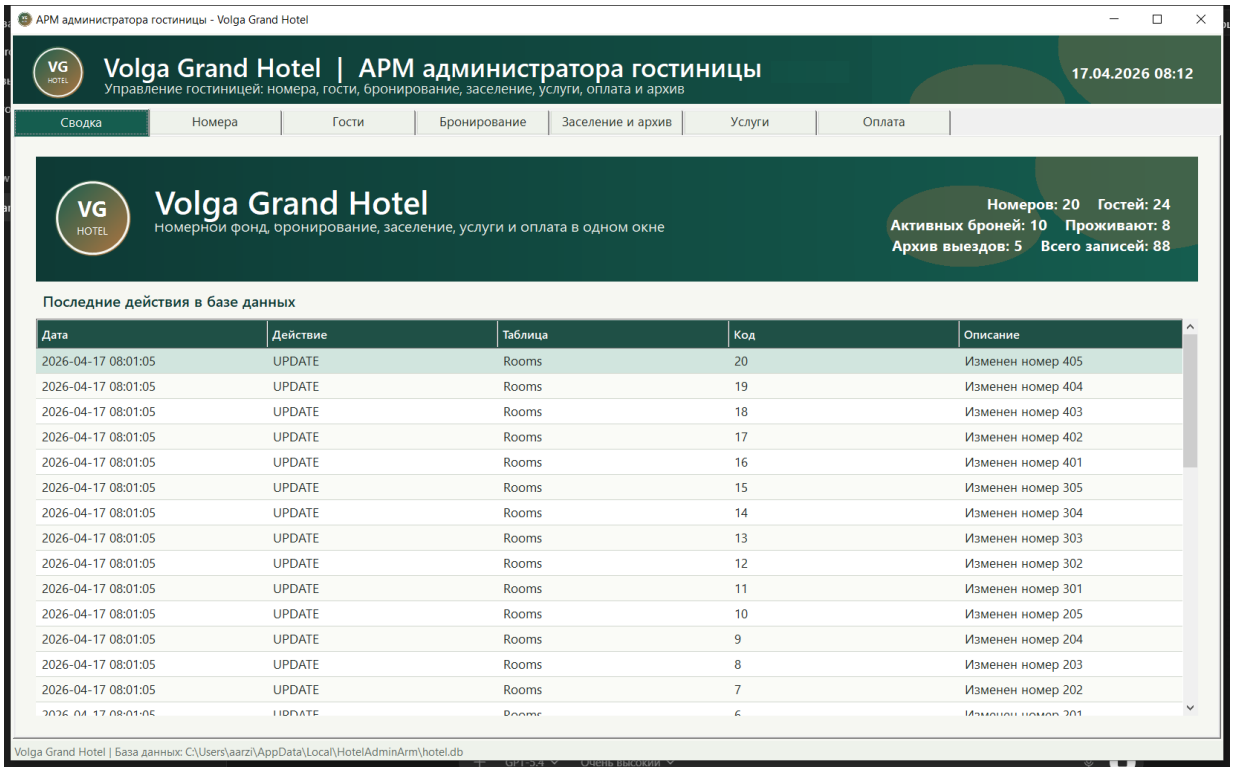


Рисунок 7 - Сводка системы и журнал действий

Сводка показывает количество номеров, гостей, активных броней, проживающих и архивных выездов. Ниже выводятся последние записи AuditLog. Проверенный демонстрационный набор содержит 88 записей в основных операционных таблицах; журнал и служебные таблицы в этот показатель не включаются.

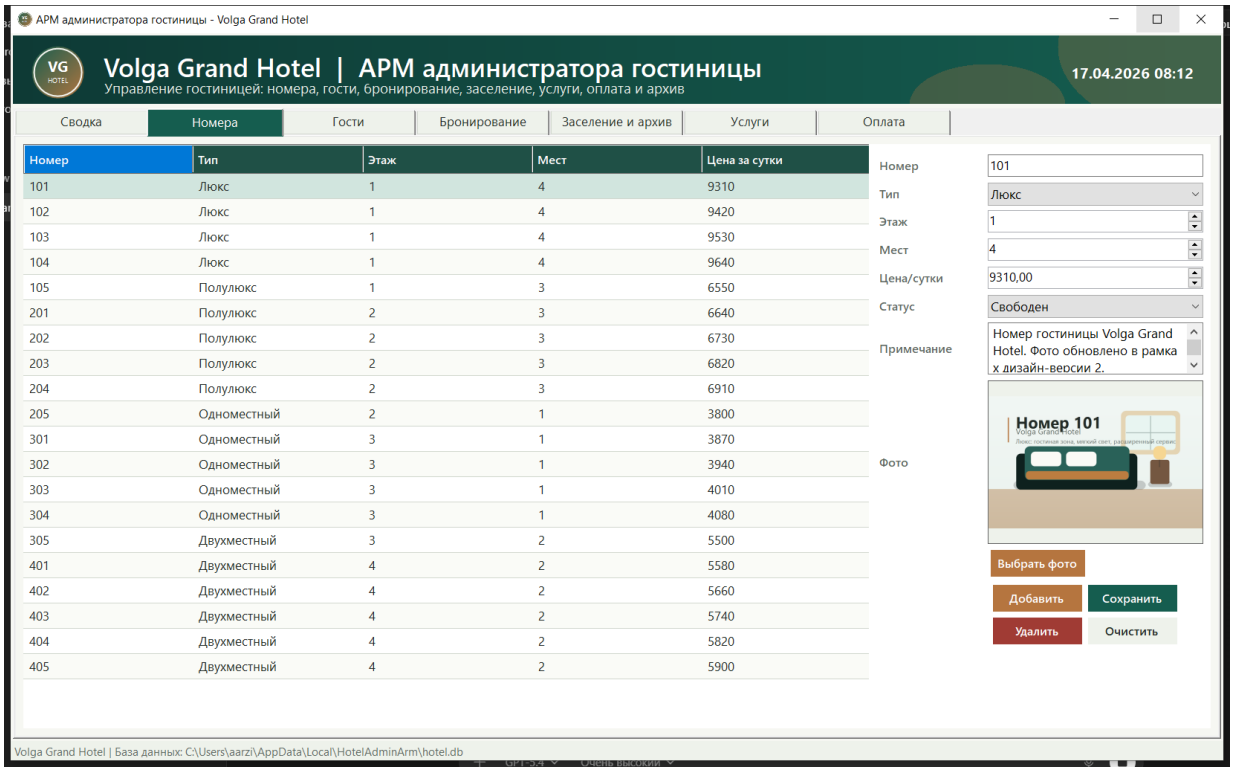


Рисунок 8 - Справочник номеров и BLOB-изображение комнаты

Во вкладке «Номера» можно редактировать номер комнаты, тип, этаж, вместимость, цену, статус, примечание и фотографию. Изображение читается из BLOB и отображается в карточке выбранной строки. Демонстрационная база содержит фотографии всех 20 номеров.

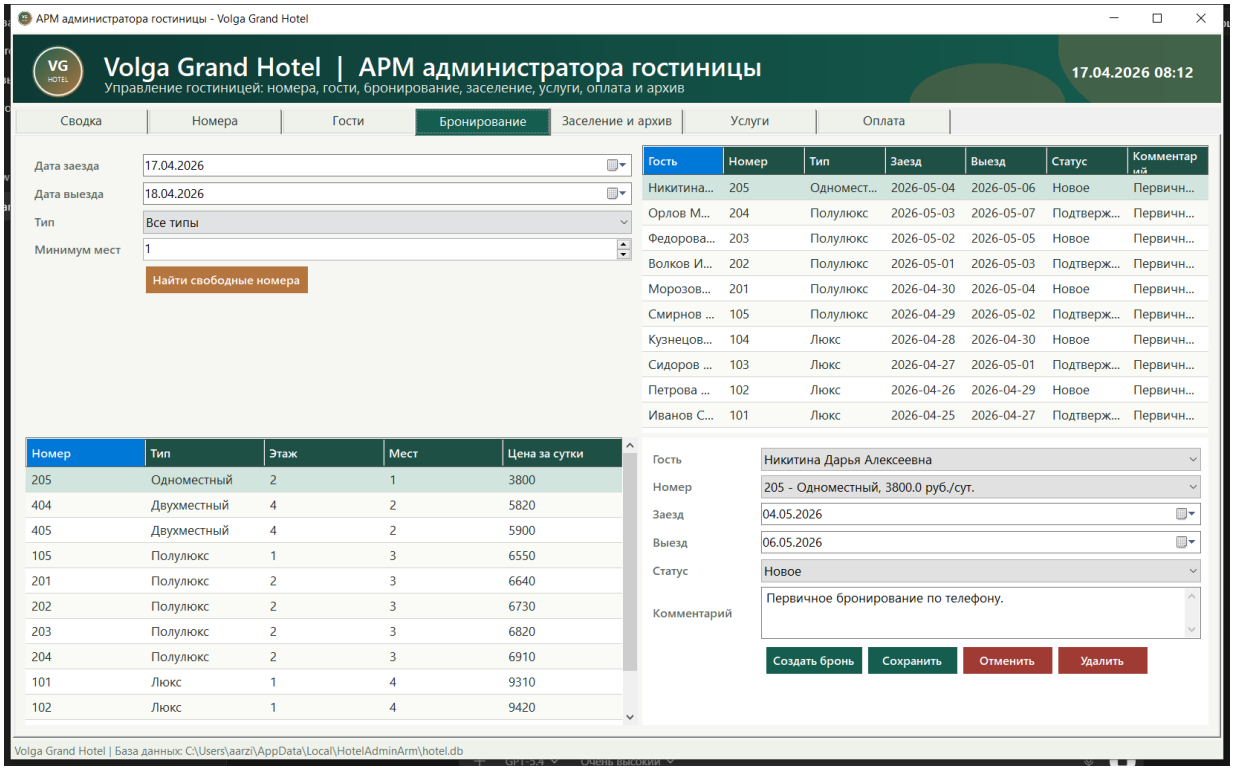


Рисунок 9 - Поиск свободных номеров и оформление бронирования

Вкладка «Бронирование» разделена на поиск свободных номеров и редактор брони. Результат поиска формируется для введённого периода, типа и вместимости. В правой части находятся список броней и поля выбранной записи, включая гостя, номер, даты, статус и комментарий.

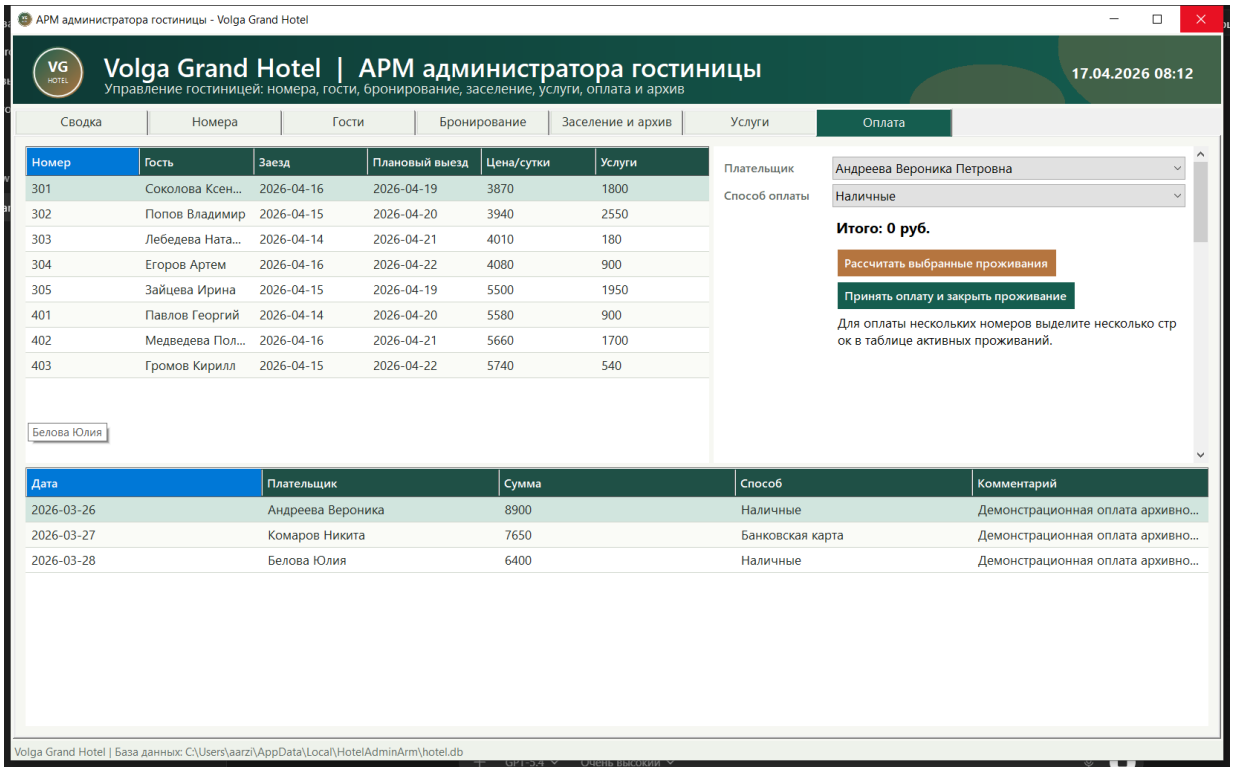


Рисунок 10 - Расчёт и оплата одного или нескольких проживаний

На вкладке «Оплата» администратор выбирает плательщика и способ расчёта, выделяет одно или несколько проживаний и получает итоговую сумму. После подтверждения создаются платёж и связи PaymentStays, выбранные проживания закрываются, а связанные номера освобождаются.

7 Тестирование системы

Тестирование выполнялось методом функциональной проверки типовых и ошибочных сценариев. Сначала проект был собран в конфигурации Release. Команда dotnet build завершилась без ошибок и предупреждений. Затем приложение было запущено, главное окно создалось с заголовком «АРМ администратора гостиницы - Volga Grand Hotel», а база hotel.db была сформирована в локальном каталоге пользователя.

Код	Сценарий	Ожидаемый результат	Итог
T1	Первый запуск без hotel.db	Созданы таблицы, триггеры и демонстрационные данные	Пройден
T2	Повторный запуск	Существующая база открывается без повторного Seed	Пройден
T3	Поиск свободных номеров	Возвращены номера без пересечений	Пройден
T4	Дата выезда не позже заезда	Операция отклонена	Пройден
T5	Пересекающаяся бронь	Триггер прерывает вставку	Пройден
T6	Заселение	Создано проживание, номер стал занятым	Пройден
T7	Добавление услуги	ServiceOrders содержит выбранную услугу и количество	Пройден
T8	Оплата нескольких проживаний	Один Payments связан с несколькими Stays	Пройден
T9	Выезд	Проживание закрыто, номер свободен, архив создан	Пройден
T10	Изменение номера и брони	AuditLog содержит запись операции	Пройден
T11	Хранение изображений	20 строк Rooms имеют непустой Photo BLOB	Пройден
T12	Объём данных	В основных таблицах 88 записей, требование 50 выполнено	Пройден

Контроль структуры базы показал наличие 12 пользовательских таблиц и 12 триггеров. В RoomTypes записано 4 типа, в Rooms - 20 номеров, в Guests - 24 гостя, в Bookings - 10 броней, в Stays - 13 проживаний, в Services - 10 услуг, в ServiceOrders - 8 заказов услуг и в Payments - 3 платежа. Пять завершённых проживаний присутствуют в GuestArchive. Все 20 номеров содержат непустое изображение.

Проверка подтверждает выполнение основных требований задания. Ограничением текущей версии является отсутствие отдельной авторизации и реального обмена с банковским шлюзом. Эти элементы присутствуют в проектных моделях как возможное расширение, но не заявляются как реализованные функции.

Заключение

В результате курсовой работы разработано автоматизированное рабочее место администратора гостиницы Volga Grand Hotel. Приложение объединяет номерной фонд, карточки гостей, бронирование, заселение, услуги, оплату, архив и аудит. База SQLite создаётся автоматически, содержит ограничения целостности, внешние ключи, 12 триггеров, демонстрационные записи и изображения комнат.

Результаты трёх лабораторных работ включены в проектирование курсовой системы. Диаграмма вариантов использования определила функции клиента и администратора. Контекстная DFD и декомпозиция уровня 0 показали основные информационные потоки и хранилища. UML-диаграмма классов сформировала объектное представление предметной области. Эти модели сопоставлены с реальными таблицами и программными модулями, поэтому пояснительная записка отражает фактическую реализацию, а не отдельный учебный пример.

Цель работы достигнута: разработана и проверена автономная информационная система, выполняющая требования задания. Дальнейшее развитие может включать аутентификацию сотрудников, печать счетов и регистрационных карт, резервное копирование, отчёты по загрузке и подключение платёжного шлюза.

Список использованной литературы

1. Тарасов С. В. СУБД для программиста. Базы данных изнутри. Москва: СОЛОН-Пресс, 2018. 320 с.
2. Прокушев Я. Е. Базы данных: учебник с практикумом. 2-е изд. Санкт-Петербург: Интермедия, 2022. 264 с.
3. Маркин А. В. СУБД «Ред База Данных». Основы SQL: учебное пособие. Москва: Ай Пи Ар Медиа, 2022. 460 с.
4. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению.
5. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы.
6. Microsoft Learn. Windows Forms documentation. URL: https://learn.microsoft.com/dotnet/desktop/winforms/
7. Microsoft Learn. Microsoft.Data.Sqlite overview. URL: https://learn.microsoft.com/dotnet/standard/data/sqlite/
8. SQLite Documentation. CREATE TABLE, foreign keys and CREATE TRIGGER. URL: https://www.sqlite.org/docs.html

Приложение А Диаграммы проектирования

В приложении собраны графические модели, подготовленные в лабораторных работах и использованные при проектировании курсовой системы. В основном тексте приведены их уменьшенные копии с пояснениями.

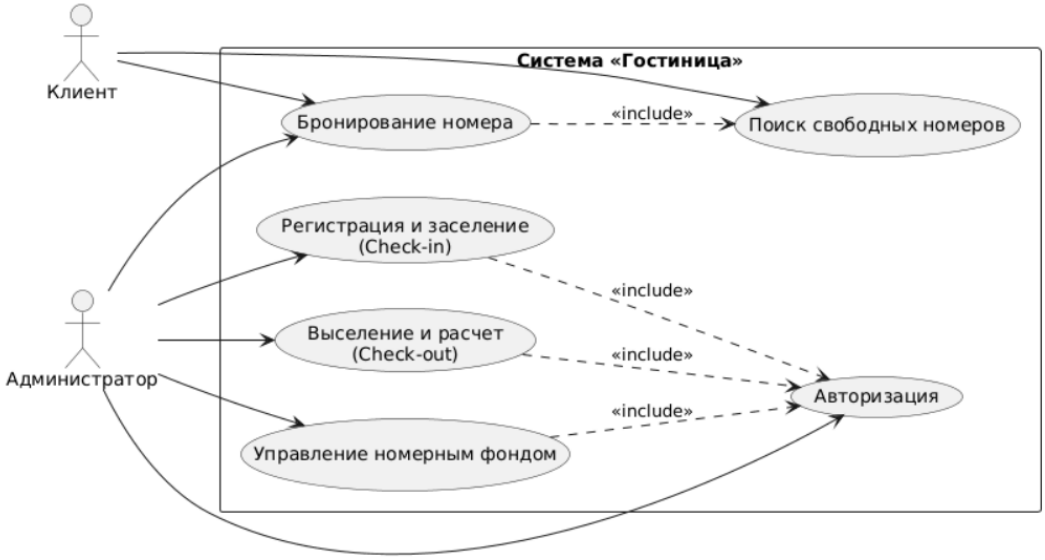


Рисунок А.1 - Диаграмма вариантов использования

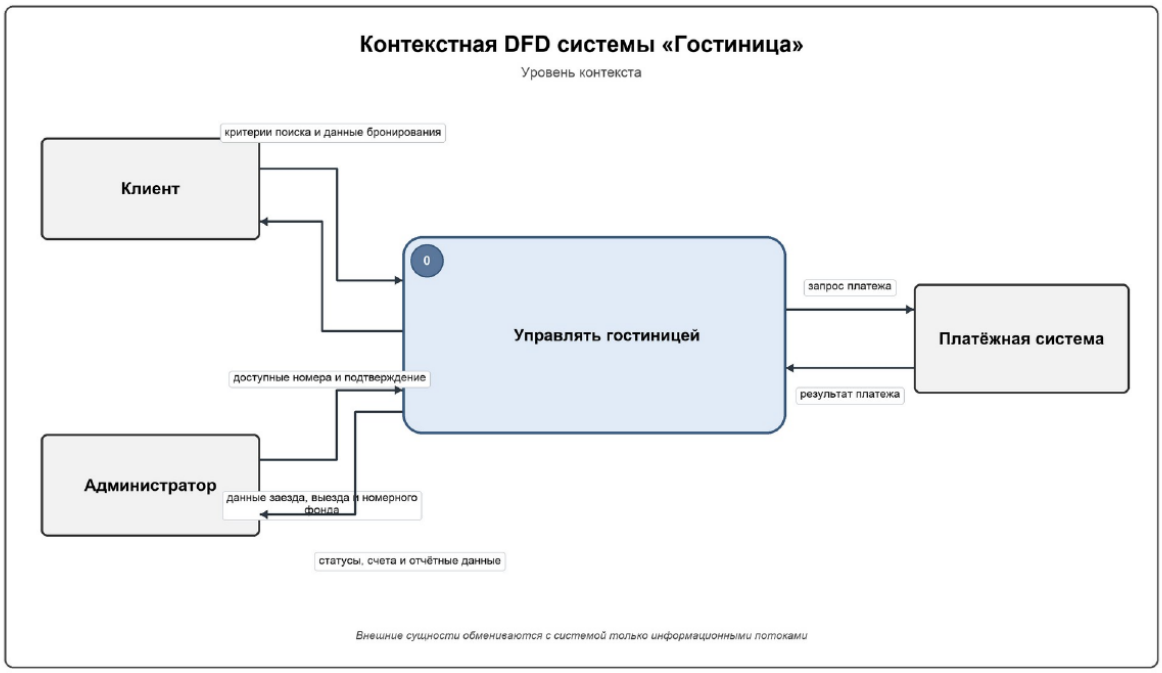


Рисунок А.2 - Контекстная DFD

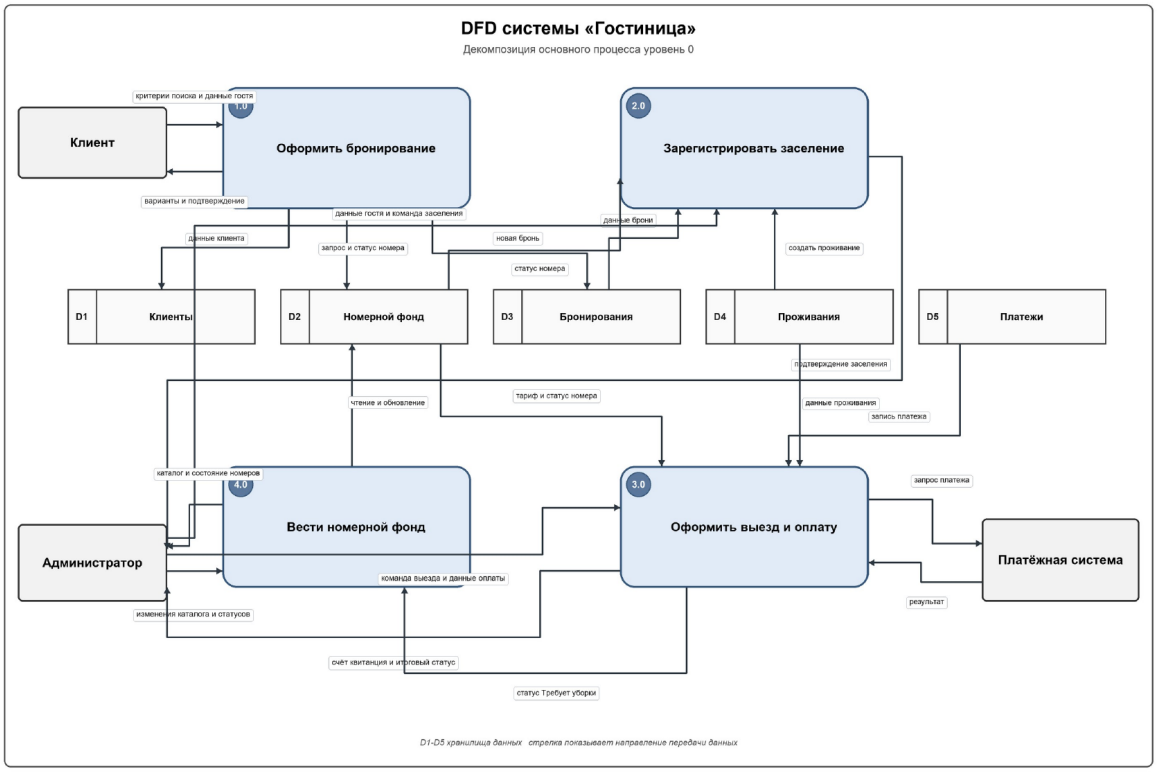


Рисунок 2 - DFD уровня 0 системы «Гостиница»

Рисунок А.3 - DFD уровня 0

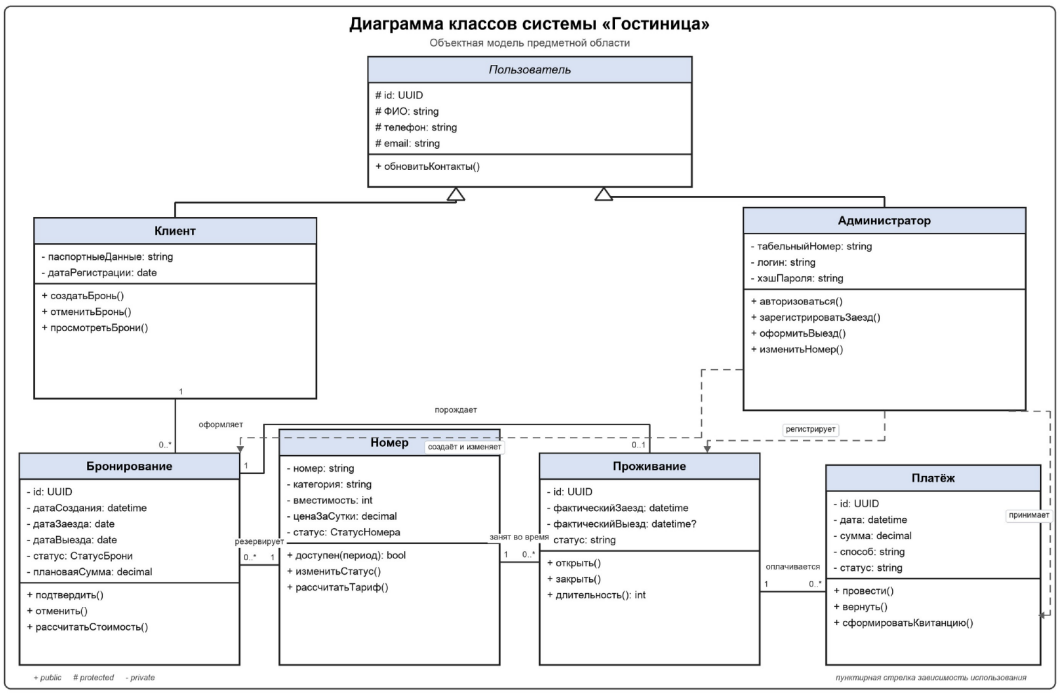


Рисунок 1 - Диаграмма классов системы «Гостиница»

Рисунок А.4 - UML-диаграмма классов



Рисунок А.5 - Концептуальная модель данных

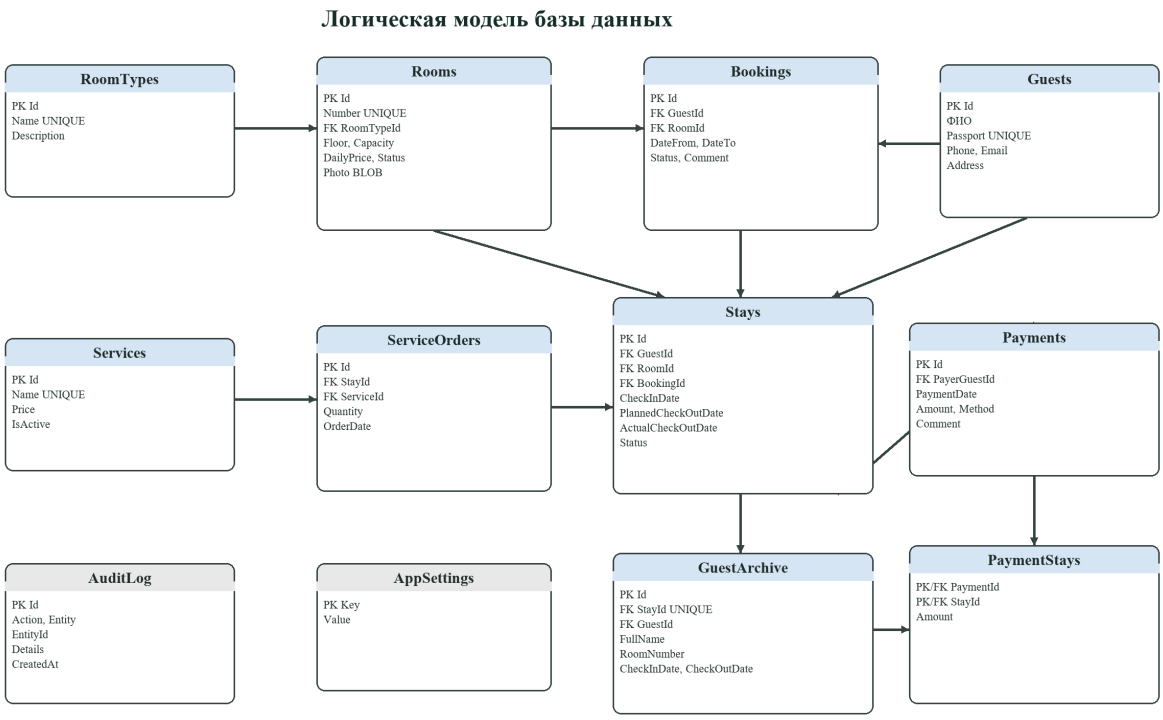


Рисунок А.6 - Логическая модель данных

Приложение Б Фрагменты текста программы

Ниже приведены ключевые файлы и фрагменты, определяющие запуск приложения, физическую схему базы данных и составные операции. Полный исходный код находится в каталоге Source/HotelAdminArm проекта.

Б.1 Файл Program.cs

```
namespace HotelAdminArm;

static class Program
{
    /// <summary>
    ///  The main entry point for the application.
    /// </summary>
    [STAThread]
    static void Main()
    {
        // To customize application configuration such as set high DPI settings or default font,
        // see https://aka.ms/applicationconfiguration.
        ApplicationConfiguration.Initialize();
        Data.Database.Initialize();
        Application.Run(new MainForm());
    }
}
```

Б.2 Файл HotelAdminArm.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net8.0-windows</TargetFramework>
    <Nullable>enable</Nullable>
    <UseWindowsForms>true</UseWindowsForms>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Data.Sqlite" Version="10.0.6" />
  </ItemGroup>

</Project>
```

Б.3 Физическая схема и триггеры Database.Schema.cs

```
namespace HotelAdminArm.Data;

internal static partial class Database
{
    private const string SchemaSql = """
        CREATE TABLE IF NOT EXISTS RoomTypes (
            Id INTEGER PRIMARY KEY AUTOINCREMENT,
            Name TEXT NOT NULL UNIQUE,
            Description TEXT NOT NULL
        );

        CREATE TABLE IF NOT EXISTS Rooms (
            Id INTEGER PRIMARY KEY AUTOINCREMENT,
            Number TEXT NOT NULL UNIQUE,
            RoomTypeId INTEGER NOT NULL,
            Floor INTEGER NOT NULL,
            Capacity INTEGER NOT NULL,
            DailyPrice REAL NOT NULL CHECK (DailyPrice >= 0),
            Status TEXT NOT NULL DEFAULT 'Свободен' CHECK (Status IN ('Свободен', 'Занят', 'Ремонт')),
            Notes TEXT,
            Photo BLOB,
            CreatedAt TEXT NOT NULL DEFAULT (datetime('now', 'localtime')),
            FOREIGN KEY (RoomTypeId) REFERENCES RoomTypes(Id)
        );

        CREATE TABLE IF NOT EXISTS Guests (
            Id INTEGER PRIMARY KEY AUTOINCREMENT,
            LastName TEXT NOT NULL,
            FirstName TEXT NOT NULL,
            MiddleName TEXT,
            BirthDate TEXT,
            Passport TEXT NOT NULL UNIQUE,
            Phone TEXT,
            Email TEXT,
            Address TEXT,
            CreatedAt TEXT NOT NULL DEFAULT (datetime('now', 'localtime'))
        );

        CREATE TABLE IF NOT EXISTS Bookings (
            Id INTEGER PRIMARY KEY AUTOINCREMENT,
            GuestId INTEGER NOT NULL,
            RoomId INTEGER NOT NULL,
            DateFrom TEXT NOT NULL,
            DateTo TEXT NOT NULL,
            Status TEXT NOT NULL DEFAULT 'Новое' CHECK (Status IN ('Новое', 'Подтверждено', 'Заселен', 'Отменено')),
            Comment TEXT,
            CreatedAt TEXT NOT NULL DEFAULT (datetime('now', 'localtime')),
            FOREIGN KEY (GuestId) REFERENCES Guests(Id) ON DELETE RESTRICT,
            FOREIGN KEY (RoomId) REFERENCES Rooms(Id) ON DELETE RESTRICT
        );
```

```
CREATE TABLE IF NOT EXISTS Stays (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    GuestId INTEGER NOT NULL,
    RoomId INTEGER NOT NULL,
    BookingId INTEGER,
    CheckInDate TEXT NOT NULL,
    PlannedCheckOutDate TEXT NOT NULL,
    ActualCheckOutDate TEXT,
    Status TEXT NOT NULL DEFAULT 'Проживает' CHECK (Status IN ('Проживает', 'Выбыл')),
    Notes TEXT,
    CreatedAt TEXT NOT NULL DEFAULT (datetime('now', 'localtime')),
    FOREIGN KEY (GuestId) REFERENCES Guests(Id) ON DELETE RESTRICT,
    FOREIGN KEY (RoomId) REFERENCES Rooms(Id) ON DELETE RESTRICT,
    FOREIGN KEY (BookingId) REFERENCES Bookings(Id) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS Services (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL UNIQUE,
    Price REAL NOT NULL CHECK (Price >= 0),
    IsActive INTEGER NOT NULL DEFAULT 1 CHECK (IsActive IN (0, 1))
);

CREATE TABLE IF NOT EXISTS ServiceOrders (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    StayId INTEGER NOT NULL,
    ServiceId INTEGER NOT NULL,
    Quantity INTEGER NOT NULL CHECK (Quantity > 0),
    OrderDate TEXT NOT NULL,
    FOREIGN KEY (StayId) REFERENCES Stays(Id) ON DELETE CASCADE,
    FOREIGN KEY (ServiceId) REFERENCES Services(Id) ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS Payments (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    PayerGuestId INTEGER NOT NULL,
    PaymentDate TEXT NOT NULL,
    Amount REAL NOT NULL CHECK (Amount >= 0),
    Method TEXT NOT NULL,
    Comment TEXT,
    CreatedAt TEXT NOT NULL DEFAULT (datetime('now', 'localtime')),
    FOREIGN KEY (PayerGuestId) REFERENCES Guests(Id) ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS PaymentStays (
    PaymentId INTEGER NOT NULL,
    StayId INTEGER NOT NULL,
    Amount REAL NOT NULL CHECK (Amount >= 0),
    PRIMARY KEY (PaymentId, StayId),
    FOREIGN KEY (PaymentId) REFERENCES Payments(Id) ON DELETE CASCADE,
    FOREIGN KEY (StayId) REFERENCES Stays(Id) ON DELETE RESTRICT
```

```
);

CREATE TABLE IF NOT EXISTS GuestArchive (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    StayId INTEGER NOT NULL UNIQUE,
    GuestId INTEGER NOT NULL,
    FullName TEXT NOT NULL,
    RoomNumber TEXT NOT NULL,
    CheckInDate TEXT NOT NULL,
    CheckOutDate TEXT NOT NULL,
    CreatedAt TEXT NOT NULL,
    FOREIGN KEY (StayId) REFERENCES Stays(Id) ON DELETE RESTRICT,
    FOREIGN KEY (GuestId) REFERENCES Guests(Id) ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS AuditLog (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    Action TEXT NOT NULL,
    Entity TEXT NOT NULL,
    EntityId INTEGER,
    Details TEXT,
    CreatedAt TEXT NOT NULL DEFAULT (datetime('now', 'localtime'))
);

CREATE TABLE IF NOT EXISTS AppSettings (
    Key TEXT PRIMARY KEY,
    Value TEXT NOT NULL
);

CREATE TRIGGER IF NOT EXISTS trg_bookings_validate_insert
BEFORE INSERT ON Bookings
BEGIN
    SELECT CASE
        WHEN julianday(NEW.DateTo) <= julianday(NEW.DateFrom)
        THEN RAISE(ABORT, 'Дата выезда должна быть позже даты заезда')
    END;

    SELECT CASE
        WHEN NEW.Status IN ('Новое', 'Подтверждено')
             AND EXISTS (
                 SELECT 1 FROM Bookings b
                 WHERE b.RoomId = NEW.RoomId
                   AND b.Status IN ('Новое', 'Подтверждено')
                   AND julianday(NEW.DateFrom) < julianday(b.DateTo)
                   AND julianday(NEW.DateTo) > julianday(b.DateFrom)
             )
        THEN RAISE(ABORT, 'Номер уже забронирован на указанный период')
    END;

    SELECT CASE
        WHEN NEW.Status IN ('Новое', 'Подтверждено')
             AND EXISTS (
                 SELECT 1 FROM Stays s
```

```
                WHERE s.RoomId = NEW.RoomId
                  AND s.Status = 'Проживает'
                  AND julianday(NEW.DateFrom) < julianday(s.PlannedCheckOutDate)
                  AND julianday(NEW.DateTo) > julianday(s.CheckInDate)
            )
        THEN RAISE(ABORT, 'Номер занят проживающим гостем на указанный период')
    END;
END;

CREATE TRIGGER IF NOT EXISTS trg_bookings_validate_update
BEFORE UPDATE ON Bookings
BEGIN
    SELECT CASE
        WHEN julianday(NEW.DateTo) <= julianday(NEW.DateFrom)
        THEN RAISE(ABORT, 'Дата выезда должна быть позже даты заезда')
    END;

    SELECT CASE
        WHEN NEW.Status IN ('Новое', 'Подтверждено')
             AND EXISTS (
                 SELECT 1 FROM Bookings b
                 WHERE b.Id <> NEW.Id
                   AND b.RoomId = NEW.RoomId
                   AND b.Status IN ('Новое', 'Подтверждено')
                   AND julianday(NEW.DateFrom) < julianday(b.DateTo)
                   AND julianday(NEW.DateTo) > julianday(b.DateFrom)
             )
        THEN RAISE(ABORT, 'Номер уже забронирован на указанный период')
    END;

    SELECT CASE
        WHEN NEW.Status IN ('Новое', 'Подтверждено')
             AND EXISTS (
                 SELECT 1 FROM Stays s
                 WHERE s.RoomId = NEW.RoomId
                   AND s.Status = 'Проживает'
                   AND julianday(NEW.DateFrom) < julianday(s.PlannedCheckOutDate)
                   AND julianday(NEW.DateTo) > julianday(s.CheckInDate)
             )
        THEN RAISE(ABORT, 'Номер занят проживающим гостем на указанный период')
    END;
END;

CREATE TRIGGER IF NOT EXISTS trg_stays_validate_insert
BEFORE INSERT ON Stays
BEGIN
    SELECT CASE
        WHEN julianday(NEW.PlannedCheckOutDate) <= julianday(NEW.CheckInDate)
        THEN RAISE(ABORT, 'Плановая дата выезда должна быть позже даты заезда')
    END;

    SELECT CASE
```

```
        WHEN NEW.Status = 'Проживает'
             AND EXISTS (
                 SELECT 1 FROM Stays s
                 WHERE s.RoomId = NEW.RoomId
                   AND s.Status = 'Проживает'
                   AND julianday(NEW.CheckInDate) < julianday(s.PlannedCheckOutDate)
                   AND julianday(NEW.PlannedCheckOutDate) > julianday(s.CheckInDate)
             )
        THEN RAISE(ABORT, 'В номере уже проживает гость на указанный период')
    END;
END;

CREATE TRIGGER IF NOT EXISTS trg_stays_validate_update
BEFORE UPDATE ON Stays
BEGIN
    SELECT CASE
        WHEN julianday(NEW.PlannedCheckOutDate) <= julianday(NEW.CheckInDate)
        THEN RAISE(ABORT, 'Плановая дата выезда должна быть позже даты заезда')
    END;

    SELECT CASE
        WHEN NEW.Status = 'Проживает'
             AND EXISTS (
                 SELECT 1 FROM Stays s
                 WHERE s.Id <> NEW.Id
                   AND s.RoomId = NEW.RoomId
                   AND s.Status = 'Проживает'
                   AND julianday(NEW.CheckInDate) < julianday(s.PlannedCheckOutDate)
                   AND julianday(NEW.PlannedCheckOutDate) > julianday(s.CheckInDate)
             )
        THEN RAISE(ABORT, 'В номере уже проживает гость на указанный период')
    END;
END;

CREATE TRIGGER IF NOT EXISTS trg_rooms_after_insert
AFTER INSERT ON Rooms
BEGIN
    INSERT INTO AuditLog(Action, Entity, EntityId, Details)
    VALUES ('INSERT', 'Rooms', NEW.Id, 'Добавлен номер ' || NEW.Number);
END;

CREATE TRIGGER IF NOT EXISTS trg_rooms_after_update
AFTER UPDATE ON Rooms
BEGIN
    INSERT INTO AuditLog(Action, Entity, EntityId, Details)
    VALUES ('UPDATE', 'Rooms', NEW.Id, 'Изменен номер ' || NEW.Number);
END;

CREATE TRIGGER IF NOT EXISTS trg_bookings_after_insert
AFTER INSERT ON Bookings
BEGIN
    INSERT INTO AuditLog(Action, Entity, EntityId, Details)
```

```
    VALUES ('INSERT', 'Bookings', NEW.Id, 'Создано бронирование');
END;

CREATE TRIGGER IF NOT EXISTS trg_bookings_after_update
AFTER UPDATE ON Bookings
BEGIN
    INSERT INTO AuditLog(Action, Entity, EntityId, Details)
    VALUES ('UPDATE', 'Bookings', NEW.Id, 'Обновлено бронирование');
END;

CREATE TRIGGER IF NOT EXISTS trg_stays_after_insert
AFTER INSERT ON Stays
WHEN NEW.Status = 'Проживает'
BEGIN
    UPDATE Rooms SET Status = 'Занят' WHERE Id = NEW.RoomId;
    INSERT INTO AuditLog(Action, Entity, EntityId, Details)
    VALUES ('INSERT', 'Stays', NEW.Id, 'Гость заселен');
END;

CREATE TRIGGER IF NOT EXISTS trg_stays_after_update
AFTER UPDATE ON Stays
BEGIN
    UPDATE Rooms
       SET Status = 'Свободен'
     WHERE Id = OLD.RoomId
       AND NEW.Status = 'Выбыл'
       AND NOT EXISTS (
           SELECT 1 FROM Stays s
           WHERE s.RoomId = OLD.RoomId AND s.Status = 'Проживает'
       );

    INSERT OR IGNORE INTO GuestArchive (StayId, GuestId, FullName, RoomNumber, CheckInDate, CheckOutDate,
        CreatedAt)
    SELECT NEW.Id, g.Id, g.LastName || ' ' || g.FirstName || ' ' || IFNULL(g.MiddleName, ''),
           r.Number, NEW.CheckInDate, COALESCE(NEW.ActualCheckOutDate, date('now', 'localtime')),
           datetime('now', 'localtime')
    FROM Guests g
    JOIN Rooms r ON r.Id = NEW.RoomId
    WHERE g.Id = NEW.GuestId AND NEW.Status = 'Выбыл';

    INSERT INTO AuditLog(Action, Entity, EntityId, Details)
    VALUES ('UPDATE', 'Stays', NEW.Id, 'Обновлена карточка проживания');
END;

CREATE TRIGGER IF NOT EXISTS trg_services_after_update
AFTER UPDATE ON Services
BEGIN
    INSERT INTO AuditLog(Action, Entity, EntityId, Details)
    VALUES ('UPDATE', 'Services', NEW.Id, 'Обновлена услуга ' || NEW.Name);
END;

CREATE TRIGGER IF NOT EXISTS trg_payments_after_insert
AFTER INSERT ON Payments
BEGIN
```

```
            INSERT INTO AuditLog(Action, Entity, EntityId, Details)
            VALUES ('INSERT', 'Payments', NEW.Id, 'Принята оплата на сумму ' || NEW.Amount);
        END;
        """;
}
```

Б.4 Фрагменты поиска, бронирования и заселения

```
            """));
    }

    private void FindAvailableRooms()
    {
        RunSafely(() =>
        {
            SetGrid(
                _availableRoomsGrid,
                Database.Query(
                    """
                    SELECT r.Id, r.Number AS 'Номер', rt.Name AS 'Тип',
                           r.Floor AS 'Этаж', r.Capacity AS 'Мест',
                           r.DailyPrice AS 'Цена за сутки'
                    FROM Rooms r
                    JOIN RoomTypes rt ON rt.Id = r.RoomTypeId
                    WHERE r.Status <> 'Ремонт'
                      AND (@typeId = 0 OR r.RoomTypeId = @typeId)
                      AND r.Capacity >= @capacity
                      AND julianday(@dateFrom) < julianday(@dateTo)
                      AND NOT EXISTS (
                          SELECT 1 FROM Bookings b
                          WHERE b.RoomId = r.Id
                            AND b.Status IN ('Новое', 'Подтверждено')
                            AND julianday(@dateFrom) < julianday(b.DateTo)
                            AND julianday(@dateTo) > julianday(b.DateFrom)
                      )
                      AND NOT EXISTS (
                          SELECT 1 FROM Stays s
                          WHERE s.RoomId = r.Id
                            AND s.Status = 'Проживает'
                            AND julianday(@dateFrom) < julianday(s.PlannedCheckOutDate)
                            AND julianday(@dateTo) > julianday(s.CheckInDate)
                      )
                    ORDER BY r.DailyPrice, r.Number;
                    """,
                    Database.P("typeId", ComboId(_availableTypeCombo)),
                    Database.P("capacity", (int)_availableCapacityNumeric.Value),
                    Database.P("dateFrom", DateValue(_availableFromPicker)),
                    Database.P("dateTo", DateValue(_availableToPicker))));
        });
    }

    private void UseAvailableRoomForBooking()
    {
        var roomId = SelectedId(_availableRoomsGrid);
        if (roomId == 0)
        {
```

```
// ...
        _bookingCommentText.Text = Convert.ToString(row["Comment"]);
    }

    private void SaveBooking(bool update)
    {
        RunSafely(() =>
        {
            var id = SelectedId(_bookingsGrid);
            if (update && id == 0)
            {
                throw new InvalidOperationException("Выберите бронирование для изменения.");
            }

            var sql = update
                ? """
                  UPDATE Bookings
                     SET GuestId = @guestId, RoomId = @roomId, DateFrom = @dateFrom,
                         DateTo = @dateTo, Status = @status, Comment = @comment
                   WHERE Id = @id;
                  """
                : """
                  INSERT INTO Bookings (GuestId, RoomId, DateFrom, DateTo, Status, Comment)
                  VALUES (@guestId, @roomId, @dateFrom, @dateTo, @status, @comment);
                  """;

            var parameters = new List<Microsoft.Data.Sqlite.SqliteParameter>
            {
                Database.P("guestId", ComboId(_bookingGuestCombo)),
                Database.P("roomId", ComboId(_bookingRoomCombo)),
                Database.P("dateFrom", DateValue(_bookingFromPicker)),
                Database.P("dateTo", DateValue(_bookingToPicker)),
                Database.P("status", Convert.ToString(_bookingStatusCombo.SelectedItem)),
                Database.P("comment", _bookingCommentText.Text.Trim())
            };

            if (update)
            {
                parameters.Add(Database.P("id", id));
            }

            Database.Execute(sql, parameters.ToArray());
            RefreshAll();
            FindAvailableRooms();
            MessageBox.Show("Бронирование сохранено.", Text, MessageBoxButtons.OK, MessageBoxIcon.Information);
        });
    }

    private void CancelBooking()
// ...
        });
    }

    private void CheckInSelectedBooking()
```

```
    {
        RunSafely(() =>
        {
            var bookingId = SelectedId(_pendingBookingsGrid);
            if (bookingId == 0)
            {
                throw new InvalidOperationException("Выберите бронь для заселения.");
            }
            var row = FirstRow(Database.Query("SELECT * FROM Bookings WHERE Id = @id;", Database.P("id", bookingId)));
            if (row is null)
            {
                return;
            }
            Database.ExecuteInTransaction((connection, transaction) =>
            {
                Database.Insert(
                    connection,
                    transaction,
                    """
                    INSERT INTO Stays (GuestId, RoomId, BookingId, CheckInDate, PlannedCheckOutDate, Status, Notes)
                    VALUES (@guestId, @roomId, @bookingId, @checkInDate, @plannedCheckOutDate, 'Проживает', @notes);
                    """,
                    Database.P("guestId", row["GuestId"]),
                    Database.P("roomId", row["RoomId"]),
                    Database.P("bookingId", bookingId),
                    Database.P("checkInDate", DateTime.Today),
                    Database.P("plannedCheckOutDate", row["DateTo"]),
                    Database.P("notes", "Заселение выполнено на основании бронирования."));
                Database.Execute(
                    connection,
                    transaction,
                    "UPDATE Bookings SET Status = 'Заселен' WHERE Id = @id;",
                    Database.P("id", bookingId));
            });
            RefreshAll();
            FindAvailableRooms();
            MessageBox.Show("Гость заселен.", Text, MessageBoxButtons.OK, MessageBoxIcon.Information);
        });
    }
// ...
```

Б.5 Фрагменты расчёта и оплаты

```
        });
    }

    private void CalculateSelectedPayment()
    {
        var stayIds = SelectedStayIdsForPayment();
        var total = CalculateStaysTotal(stayIds, DateTime.Today);
        _paymentTotalLabel.Text = $"Итого: {total:N2} руб.";
    }

    private void PaySelectedStays()
    {
        var stayIds = SelectedStayIdsForPayment();
        PayForStays(stayIds);
    }

    private int[] SelectedStayIdsForPayment()
    {
        return _paymentStaysGrid.SelectedRows
            .Cast<DataGridViewRow>()
            .Where(row => row.Cells["Id"].Value is not null)
            .Select(row => Convert.ToInt32(row.Cells["Id"].Value))
            .Distinct()
            .ToArray();
    }

    private void PayForStays(int[] stayIds)
    {
        RunSafely(() =>
        {
            if (stayIds.Length == 0)
            {
                throw new InvalidOperationException("Выберите проживание для оплаты.");
            }

            var total = CalculateStaysTotal(stayIds, DateTime.Today);
            if (total <= 0)
            {
                throw new InvalidOperationException("Сумма оплаты должна быть больше нуля.");
            }

            if (MessageBox.Show(
                    $"Принять оплату {total:N2} руб. и закрыть выбранные проживания?",
                    Text,
                    MessageBoxButtons.YesNo,
                    MessageBoxIcon.Question) != DialogResult.Yes)
            {
                return;
// ...
            .ToArray();
    }
```

```
    private void PayForStays(int[] stayIds)
    {
        RunSafely(() =>
        {
            if (stayIds.Length == 0)
            {
                throw new InvalidOperationException("Выберите проживание для оплаты.");
            }

            var total = CalculateStaysTotal(stayIds, DateTime.Today);
            if (total <= 0)
            {
                throw new InvalidOperationException("Сумма оплаты должна быть больше нуля.");
            }

            if (MessageBox.Show(
                    $"Принять оплату {total:N2} руб. и закрыть выбранные проживания?",
                    Text,
                    MessageBoxButtons.YesNo,
                    MessageBoxIcon.Question) != DialogResult.Yes)
            {
                return;
            }

            Database.ExecuteInTransaction((connection, transaction) =>
            {
                var paymentId = Database.Insert(
                    connection,
                    transaction,
                    """
                    INSERT INTO Payments (PayerGuestId, PaymentDate, Amount, Method, Comment)
                    VALUES (@payerGuestId, @paymentDate, @amount, @method, @comment);
                    """,
                    Database.P("payerGuestId", ComboId(_paymentPayerCombo)),
                    Database.P("paymentDate", DateTime.Today),
                    Database.P("amount", total),
                    Database.P("method", Convert.ToString(_paymentMethodCombo.SelectedItem)),
                    Database.P("comment", $"Оплата проживаний: {string.Join(", ", stayIds)}"));

                foreach (var stayId in stayIds)
                {
                    var stayTotal = CalculateStayTotal(stayId, DateTime.Today, connection, transaction);
                    Database.Execute(
                        connection,
                        transaction,
// ...
            return total;
        }

        private static decimal CalculateStayTotal(
            int stayId,
            DateTime checkoutDate,
```

```
        Microsoft.Data.Sqlite.SqliteConnection? connection,
        Microsoft.Data.Sqlite.SqliteTransaction? transaction)
    {
        const string sql = """
            SELECT
                MAX(1, CAST(julianday(@checkoutDate) - julianday(s.CheckInDate) AS INTEGER)) * r.DailyPrice
                + IFNULL((
                    SELECT SUM(so.Quantity * sv.Price)
                    FROM ServiceOrders so
                    JOIN Services sv ON sv.Id = so.ServiceId
                    WHERE so.StayId = s.Id
                ), 0) AS Total
            FROM Stays s
            JOIN Rooms r ON r.Id = s.RoomId
            WHERE s.Id = @stayId;
            """;

        object value;
        if (connection is null)
        {
            value = Database.Scalar<double>(
                sql,
                Database.P("checkoutDate", checkoutDate),
                Database.P("stayId", stayId));
        }
        else
        {
            value = Database.Scalar<double>(
                connection,
                transaction,
                sql,
                Database.P("checkoutDate", checkoutDate),
                Database.P("stayId", stayId));
        }

        return Convert.ToDecimal(value);
    }
}
// ...
```